CAS (Heimdall) – program design

Inlet pump: configuration, commands, prerequisites and dataflow

Sigurd R. Brekk

October 15, 2026

1 Purpose

This document shows how the Heimdall PLC program starts and runs an inlet pump. It answers four questions:

1. which configuration values must be set before an inlet pump can start (Section 5.3);
2. which commands start, stop and reset it (Section 5.4);
3. which other systems must be available or running (Section 5.2);
4. how a value travels through the program, from the Modbus word read from the remote I/O to the speed setpoint written to the Framo controller (Section 6).

Section 7 lists where the program differs from the functional description [FD] and other issues found while tracing the dataflow.

The program was read on 2026-10-15 from the CODESYS project `heimdall-plc`, branch `inletControl`, application `PFC200_Sim`, after the merge with the outlet valve work.

Sources

ID	Document
[COO]	J. Cooling, *Software Engineering for Real-Time Systems* (The Complete Edition), Packt, 2020. §4.6.2 materials flow model, §7.2.1 context diagrams, §7.2.3 state transition diagrams, §7.2.4 functional and data flow diagrams, §14.3.3 functional diagram description.
[FD]	P-15169-CAS-FD-01 Funksjonsbeskrivelse CAS, rev. B, 13.05.26. §3.2.1 Vanninntak, §3.2.2 Regulering av vann-nivå i merden, §3.2.4 Oksygentilførsel.
[DES]	CAS (Heimdall) – design basis for control (*System Modelling/CAS design basis.tex*).
[PLC]	CODESYS project heimdall-plc, branch inletControl, application PFC200_Sim (read 2026-10-15).

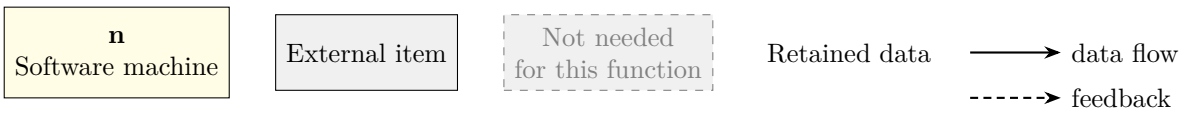
2 Notation

The diagrams follow Cooling's functionally structured diagrams [COO §7.2].

- A **context diagram** [COO §7.2.1, Fig. 7.1] shows the PLC software as one black box and the external items it exchanges data with. It defines every device the software talks to and every signal it must handle.
- A **functional flow diagram** (FFD) [COO §7.2.4, Fig. 7.8] shows the processing as a flow of material, here data, through *software machines* (SM). Each SM is a self-contained unit with one clear function; its outputs are the result of processing its inputs. The underlying model is the materials flow model [COO §4.6.2]: a value enters on the left and is worked on by one SM after the other, like a work piece on a production line.
- Since the PLC runs the SMs in sequential code, only one SM is active at a time and the order matters. The FFD is therefore accompanied by an **execution sequence** [COO Fig. 7.10] that names the code unit (task, program, function block) running each SM.
- The behaviour over time of the Framo pump state is shown in a **state transition diagram** [COO §7.2.3, Fig. 7.6].

In CODESYS terms an SM is a program (PRG), a function block (FB) instance or a method. Its *in parameters* are the variables it reads (VAR_INPUT or VAR_IN_OUT), its *out parameters* the variables it writes. Variable names are given in full from the global variable list, e.g. `GVLMain.Inlet[i].InletPump.State`; `i` = 0, 1, 2 is the inlet (inlet 1, 2, 3) and `n` = `i`+1 numbers the Framo data sources `Framo_1..3`.

Symbols used in the diagrams:



3 Context diagram

Figure 1 shows the PLC software and its surroundings as far as the inlet pumps are concerned. Table 1 gives the interface of each external item.

4 Tasks and execution sequence

The SMs run in three tasks (Task Configuration of `PFC200_Sim`). Within a task the programs run in the order listed, and within `PRG.Main` the function blocks run in the order they are called. Figure 2 is the execution sequence in the outline form of [COO Fig. 7.10].

Consequence for the timing. `IO_Processing` runs after `Main`, and `InletSupervisor` runs before `PenControl`. A level word read in cycle k is therefore scaled and averaged in cycle $k+1$ and reaches the level controller in cycle $k+2$. The speed setpoint is then handed to `FramoDataProcessing` in the next run of `OPC_Processing`, and the OPC UA client sends it with its own publishing interval. Together

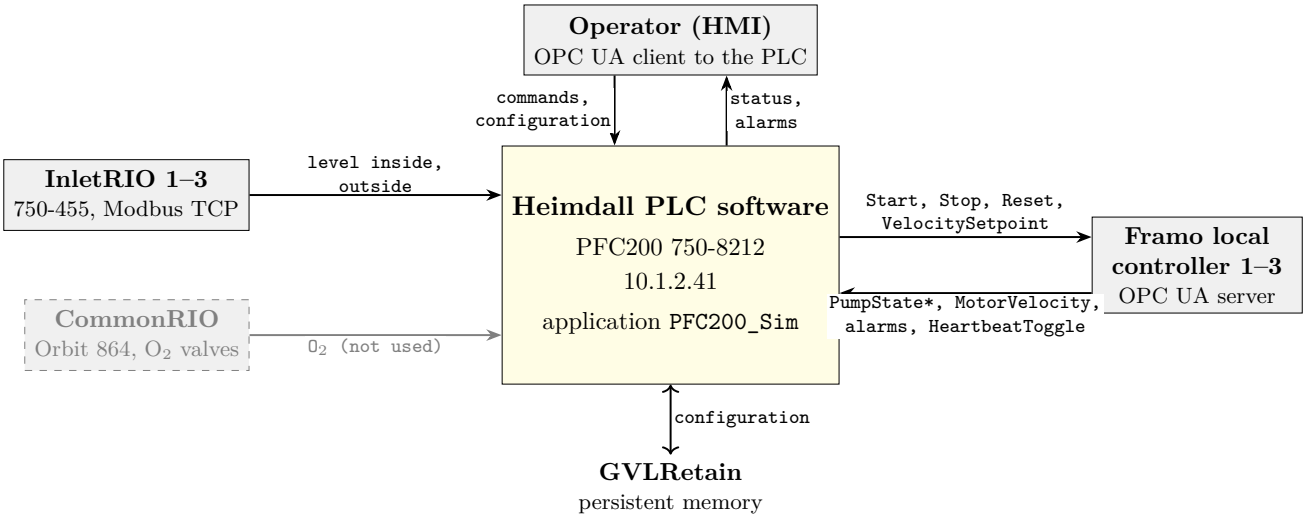


Figure 1: Context diagram of the inlet pump function [COO Fig. 7.1]. The CommonRIO carries the oxygen sensors and valves; the inlet pump does not need it.

Table 1: External items of the inlet pump function.

External item	Link	PLC code unit	Data
InletRIO 1–3	Modbus TCP, FC23, read words 0–25, write words 0–20, every 60 ms	`InletRIO[i]` in `PRG.IO_Processing`	In: level inside (word 0), level outside (word 1), oxy pump and oxynator pressure (words 2, 3), digital inputs (words 4, 5). Out: reset of field power (word 0 bit 0).
Framo local controller 1–3	OPC UA, data sources `Framo_1..3` in the `DatasourcesManager`	`PRG.FramoDataProcessing`, `Datasources_PRG`	In: pump state, motor values, alarms, heartbeat [FD Tabell 6]. Out: `Start`, `Stop`, `Reset`, `SpeedSetpoint`.
Operator (HMI)	OPC UA server of the PLC (Symbol Configuration: `GVLMain`, `GVLIO`)	–	Writes commands and configuration in `GVLMain`; reads status and alarms.
GVLRetain	persistent variables	`Main.UpdateRetain`	Configuration copied to `GVLMain` in the first cycle, and back on every change.
CommonRIO	Modbus TCP and Modbus RTU (750-652)	`CommonRIO`, `IO_Processing_RTU`	Oxygen sensors and valves. Not needed to start or run an inlet pump.

```
Task Task (cyclic)
Main
  InletSupervisor                  SM 7, 8, S1
  FOR i := 0 TO 2:
    InletControl[i]                SM 9, 10, S2
      InletSensors                   SM 2, 3, 4
    OxyControl[i]
  OutletControl
  PenControl                           SM 5, 6
  UpdateRetain
IO_Processing
  InletRIO[i], CommonRIO                SM 1
IO_Processing_RTU                (Orbit 864)
```

```
Task OPC_Processing
FramoDataProcessing    SM 11, F, S3
```

```
Task DatasourcesTask
Datasources_PRG                  SM 12
```

`PRG.Controllers` is in the project but is not called by any task.[D14]

Figure 2: Execution sequence [COO Fig. 7.10]. SM numbers refer to Figure 5 and Table 7; S1–S3 and F to Figure 4.

with the 60 ms Modbus poll this is some tenths of a second, small next to the 2 s dead time of the level loop [DES §7].

5 Starting an inlet pump

5.1 Pump state

The Framo local controller owns the pump state. It publishes one Boolean per state (`PumpStateAvailable`, `PumpStateStart`, ...); `FramoDataProcessing` turns these into the enumeration `GVLMain.Inlet[i].InletPump.State` (`ENUM.InletPumpState`). The PLC only asks for transitions with `Start`, `Stop` and `Reset`. Figure 3 is the state transition diagram as described in [FD Tabell 5]; Table 2 shows what the PLC does in each state (`InletControl.InletPumpControl`).

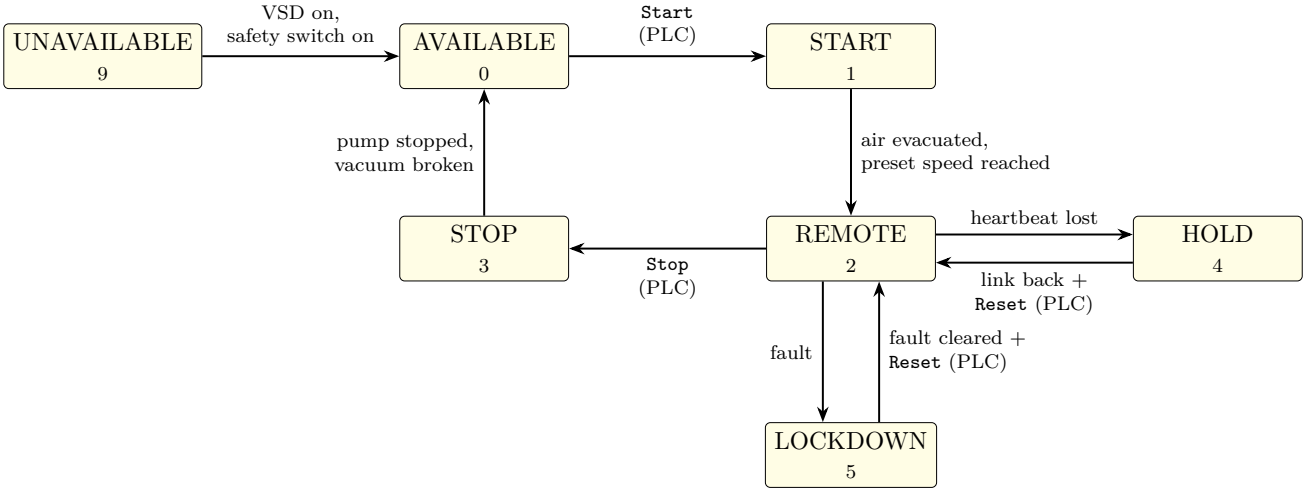


Figure 3: State transition diagram of the Framo inlet pump [FD Tabell 5], [COO §7.2.3]. Numbers are the values of `ENUM.InletPumpState`. Transitions not described in [FD] (e.g. from LOCKDOWN after `Reset` when the pump has stopped) are set by the Framo controller.

Table 2: PLC action per pump state (`InletControl.InletPumpControl`).

State	`Output.VelocitySetpoint`	Start allowed	Running	In LIC01	Comment
UNAVAILABLE	0	no	no	no	Also shown while the Framo link is down (no heartbeat answer).[D9]
AVAILABLE	`Input.MotorVelocity`	yes	no	no	Setpoint follows the actual speed, as [FD] asks.
START	`Input.MotorVelocity`	yes	yes	yes	LIC01 tracks the mean speed of the starting pumps (SM 7).
REMOTE	`InletPumpSpeedSetpoint` (LIC01 output in auto, `Command.Setpoint` in manual)	yes	yes	yes	The only state in which the PLC sets the speed.
HOLD	`Input.MotorVelocity`	yes	yes*	no*	Framo runs a preset speed. LIC01 is frozen until the pump is back in REMOTE.[D3]
STOP	0	no	yes	yes	
LOCKDOWN	0	no	no	no	Needs `Reset`.

* The local variable `Running` is set TRUE in the HOLD branch, but `Status.EnableLevelControl` is computed before the CASE from the states START, REMOTE and STOP only. `Start allowed` = `StartAvailable`; `In LIC01` = `Status.EnableLevelControl` with the pump in auto.

5.2 Other systems that must be available or running

Table 3 lists what must be in place before `Start` reaches the pump and the level controller can take over. Only the first four rows are needed for the pump to start. The rest are needed for level control.

Table 3: Prerequisites for starting an inlet pump.

System	What must hold	Where the PLC sees it
Framo local controller n	Powered, VSD on, safety switch on, no emergency stop, no fault; vacuum system ready (handled by the local controller). Pump state AVAILABLE.	`Inlet[i].InletPump.State = Available`
OPC UA link to Framo n	Data source `Framo_n` connected, so that commands are written and the state is read; Framo answers the heartbeat. Without it the PLC shows the pump UNAVAILABLE and sends no Start.[D9]	`Inlet[i].InletPump.Status.CommunicationOK`
PLC application	Running, with the tasks `Task`, `OPC_Processing` and `DatasourcesTask`.	–
Operator (HMI)	Connected to the PLC's OPC UA server to write the commands.	–
InletRIO i	Connected (`GVLIO.InletRIO[i].Status.CommOK`); otherwise the level words are not updated. Not an interlock for the start.	`InletRIO[i].Status.CommOK`
Level transmitters LT inside/outside	At least one valid pair: both transmitters of an inlet 4–20 mA healthy (status bits 1..0 = 00) and enabled, and the pair enabled. Without one, LIC01 runs the pumps at `SafeOutput` and raises *LevelMeasurementLost*. Not an interlock for the start.	`Pen.Status.LevelControlOK`
Oxygen system, outlet valve, CommonRIO	Not needed. In [FD] the oxygen system depends on the inlet pump, not the other way round: the oxy pump runs only when the inlet pump on the same line runs [FD §3.2.4].	–

5.3 Configuration

Table 4 lists the configuration values the inlet pump function uses. All but the data source endpoints live in `GVLMain`, are written from the HMI and are kept in `GVLRetain` by `Main.UpdateRetain`. Defaults are those in `GVL.ConfigValues` of `PFC200_Sim`, applied on a fresh download.

5.4 Commands

The commands are variables in `GVLMain` that the HMI sets. Commands marked *pulse* are reset by the PLC once acted on. Table 5 gives the commands; the steps below give the order.

Automatic start of the pen (normal operation).

1. Put each pump that shall run in auto: `Inlet[i].InletPump.Command.Auto` (default: auto).
2. Set the level setpoint `Pen.Command.LevelSetpoint` [m].[D13]

Table 4: Configuration for the inlet pump function. *Needed for*: S = pump start, L = level control.

Variable	Default	For	Meaning, and what happens if it is wrong
Data sources `Framo_1..3` (DatasourcesManager)	per project	S	OPC UA endpoint of each Framo controller. Wrong: no commands reach the pump.
`InletRIOConfig[i].Enabled`	TRUE	L	Connects the Modbus TCP master.
`InletRIOConfig[i].IP`, `.Port`	10.1.2.101, 5121–5123 (simulator)	L	Address of InletRIO i. IP 0 or port 0 raises *MissingIPaddress*.
`Inlet[i].Input.LevelInsidePen.Config.Enable`	TRUE	L	Transmitter in use; disabled, its pair drops out.
`Inlet[i].Input.LevelInsidePen.Config.MinValue`, `.MaxValue`	0.0, 2.0 m	L	Range for 4 and 20 mA.
`Inlet[i].Input.LevelInsidePen.Config.QualityEnabled`	TRUE	L	Quality alarm on a bad transmitter (after `QualityDelay`, 1 s).
`Inlet[i].Input.LevelInsidePen.Config.FilterFc`	0.05 Hz	–	Display smoothing only; not in the control path.
`Inlet[i].Input.LevelOutsidePen.Config.*`	as inside	L	Same for the outside transmitter.
`Inlet[i].Input.PenOverHeight.Config.Enable`	TRUE	L	The inlet's level pair is used in LIC01's process value.
`Inlet[i].Input.PenOverHeight.Config.FilterFc`	0.05 Hz	L	Cut-off of the pair's Butterworth2 low-pass; 0 turns it off.
`PenConfig.LevelController.Kp`	450 rpm/m	L	Gain of LIC01.
`PenConfig.LevelController.Ti`	600 s[D11]	L	Integral time.
`PenConfig.LevelController.Td`	0 s	L	Derivative time (0 = PI).
`PenConfig.LevelController.MinOutput`, `.MaxOutput`	40, 130 rpm	L	Speed range of the pumps [FD Tabell 8]; also the PID's output limits.
`PenConfig.LevelController.SafeOutput`	60 rpm	L	Pump speed while no valid level pair is left.
`InletConfig[i].HeartbeatInterval`, `.HeartbeatTimeout`	1 s, 5 s	S	Framo heartbeat: the PLC sets the shared bit 1 s after Framo cleared it; alarm when Framo has not cleared it for 5 s.
`InletConfig[i].StartupTime`, `.ShutdownTime`	30 s, 30 s	S	Longest time from Start (Stop) until Framo reports START (STOP); then the start-up (stop) timeout alarm.[D8]

3. Leave LIC01 in auto (`LevelController.Command.Manual` = FALSE, default).
4. Give `Pen.Command.Start`. Every pump in auto gets `Start`; while a pump is in START, LIC01 follows its speed, and when the first pump reaches REMOTE LIC01 takes over bumplessly.
5. `Pen.Command.Stop` stops every pump in auto.

Manual start of one pump.

1. `Inlet[i].InletPump.Command.Manual`; the pump leaves level control.
2. Set the speed `Inlet[i].InletPump.Command.Setpoint` [rpm]; used in REMOTE.
3. `Inlet[i].InletPump.Command.Start`; stop with `Inlet[i].InletPump.Command.Stop`.

Table 5: Commands for the inlet pumps.

Command	Type	Effect
`Pen.Command.Start`	pulse	Latches `PenStart` in `InletSupervisor`; every pump in auto gets `Inlet[i].Command.AutoStartInletPump`. Reset by the PLC when a pump runs.
`Pen.Command.Stop`	pulse	Clears `PenStart`; every pump in auto gets `AutoStopInletPump`.
`Pen.Command.LevelSetpoint`	value [m]	Setpoint of LIC01 (overheight inside the pen). [FD] gives it in cm.
`Inlet[i].InletPump.Command.Manual` / `.Auto`	pulse	Sets / clears `InletPump.Status.Manual`.
`Inlet[i].InletPump.Command.Start` / `.Stop`	pulse	Start or stop in manual.
`Inlet[i].InletPump.Command.Setpoint`	value [rpm]	Speed in manual, sent in REMOTE.
`Inlet[i].InletPump.Command.Reset`	level	Sent to Framo as `Reset` (HOLD, LOCKDOWN).
`LevelController.Command.Manual` / `.Auto`	level / pulse	LIC01 manual or auto.
`LevelController.Command.ManualOutput`	value [rpm]	LIC01 output in manual.

5.5 Start command path

Figure 4 follows the start command from the HMI to the Framo controller and the pump state back. Table 6 names the variables.

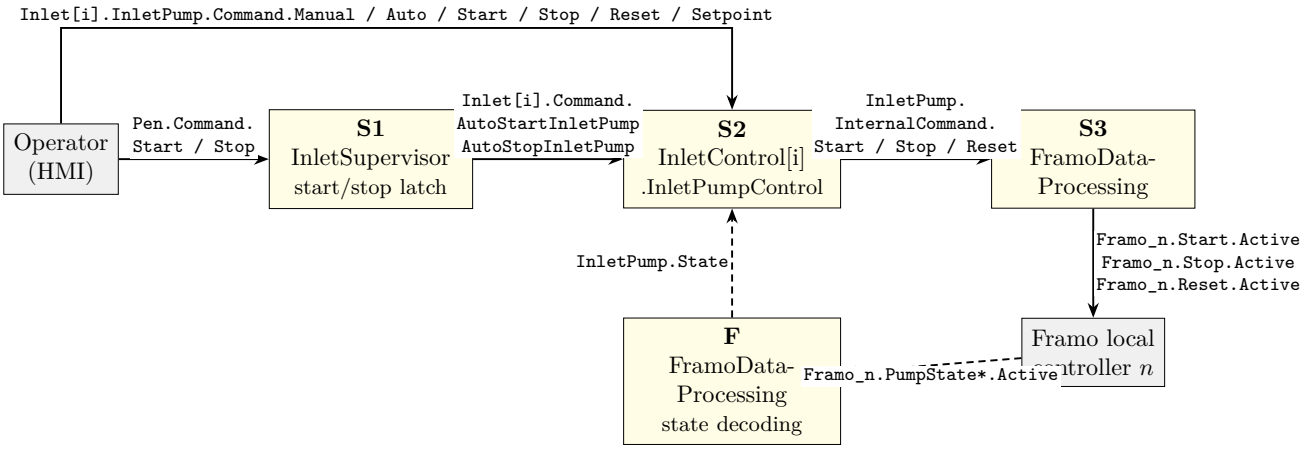


Figure 4: FFD of the start command [COO Fig. 7.8]. The manual commands enter at S2.

Table 6: Start command dataflow.

SM	Code unit	Reads	Operation and writes
S1	`InletSupervisor` (body)	`Pen.Command.Start`, `Pen.Command.Stop`, `Inlet[i].InletPump.Status.Manual`	`PenStart := (PenStart OR Start) AND NOT Stop`. For each pump not in manual: `Inlet[i].Command.AutoStartInletPump := PenStart`, `AutoStopInletPump := NOT PenStart`.
S2	`InletControl[i].InletPumpControl`	`AutoStartInletPump`, `AutoStopInletPump` (auto) or `InletPump.Command.Start/Stop` (manual); `InletPump.State`	`StartPump` latches the start until a stop; `StopRequest` = stop command. `InletPump.Output.Start := StartPump AND StartAvailable`. The body copies to `InletPump.InternalCommand.Start`, `.Stop := StopRequest`, `.Reset := Command.Reset`.
S3	`FramoDataProcessing`	`Inlet[i].InletPump.InternalCommand.*`	`Framo_n.Start.Active`, `Framo_n.Stop.Active`, `Framo_n.Reset.Active`; the Framo controller acts on the rising edge.
F	`FramoDataProcessing`	`Framo_n.PumpStateUnavailable.Active` ... `PumpStateLockdown.Active`	While `Inlet[i].InletPump.Status.CommunicationOK` (set by `InletControl` from the heartbeat): the first TRUE flag sets `Inlet[i].InletPump.State`. Otherwise `State:=Unavailable`.[D9]

6 Dataflow: from level transmitter to pump speed

Figure 5 is the functional flow diagram of the level loop, the main dataflow of the inlet pump: the two level words read from InletRIO i are scaled, their difference (the inlet's level pair) is filtered, the valid pairs are averaged over the inlets, the result is compared with the setpoint in LIC01, and the controller output is sent as the speed setpoint to every pump in level control. Table 7 names, for every SM, the variables it reads, what it does with them and the variables it writes.

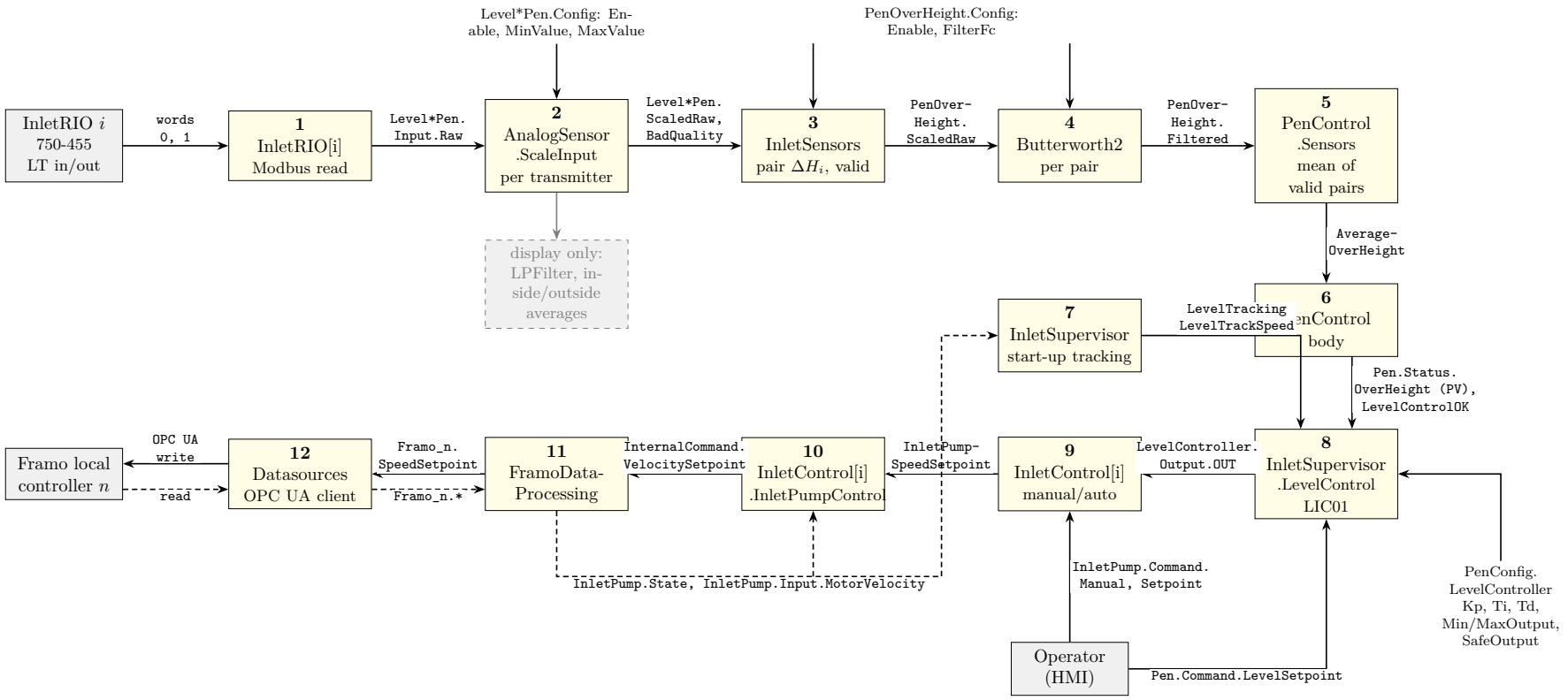


Figure 5: Functional flow diagram of the level loop, one inlet shown [COO Fig. 7.8]. SMs 1–4, 9 and 10 run once per inlet; SMs 5–8 once for the pen. Solid: data flow; dashed: feedback from the pump. Level*Pen stands for LevelInsidePen and LevelOutsidePen. A pair with a lost transmitter drops out at SM 3; with no valid pair, SM 8 runs LIC01 at SafeOutput. Execution order in Figure 2.

Table 7: Dataflow of the level loop. Unit after each SM in square brackets. "Main." and "GVLMain." prefixes are left out where the context is clear.

SM	Code unit (task)	Reads	Operation	Writes
1	`IO_Processing.InletRIO[i]` (Task)	Modbus TCP response `TCPResponse.awData[0..25]`	`WagoAppPlcModbus.FbMbMasterTcp`, function code 23, reads 26 words from address 0 every 60 ms (`TCPDelay`). `InputMapping` copies the words only while `Status.CommOK`; otherwise the last value stays.	`GVLMain.Inlet[i].Input.LevelInsidePen.Input.Raw` := word 0; `GVLMain.Inlet[i].Input.LevelOutsidePen.Input.Raw` := word 1 [counts]
2	`InletControl[i].InletSensors.LevelInsidePen.ScaleInput` (Task); same for `LevelOutsidePen`	`Input.Raw`, `Config.MinValue`, `Config.MaxValue`	750-455 word: value in bits 14..4, reserved bits 3..2, diagnostics in bits 1..0. `Quality := Raw AND 2#11`; `Raw := Raw AND 16#FFF8` (written back to the GVL).[D12] If `Quality = 0`: ScaledRaw = Raw · (Max − Min)/32760 + Min, `BadQuality := FALSE`; else `ScaledRaw := Min`, `BadQuality := TRUE`, and the transmitter's Quality alarm (`Config.QualityEnabled`, 1 s). The per-transmitter `LPFilter` (`SCL.Butterworth`) gives `.Filtered` for display only.	`Inlet[i].Input.LevelInsidePen.ScaledRaw` [m], `.Status.BadQuality`
3	`InletControl[i].InletSensors` (Task)	`LevelInsidePen.ScaledRaw`, `LevelOutsidePen.ScaledRaw`, their `.Config.Enable` and `.Status.BadQuality`, `PenOverHeight.Config.Enable`	Level pair of the inlet [FD §3.2.2]: ΔH_i = inside − outside from the unfiltered scaled values. The pair is valid when both transmitters are enabled and healthy and the pair is enabled; if either transmitter is lost, the whole pair drops out.	`Inlet[i].Input.PenOverHeight.ScaledRaw` [m], `.Status.BadQuality` (= not valid)
4	`InletSensors.OverHeightFilter` (`Butterworth2`) (Task)	`PenOverHeight.ScaledRaw`, `PenOverHeight.Config.FilterFc`	Second-order Butterworth low-pass on the pair's ΔH_i, LREAL, bilinear with pre-warping. Sample time: the calling task's cycle time (`GetTaskIntervalUs`, `CmpIecTask`). Restarts at the current value when the pair becomes valid (and at start), so a returning pair causes no dip.	`Inlet[i].Input.PenOverHeight.Filtered` [m] (0 while not valid)
5	`PenControl.Sensors` (Task)	`Inlet[0..2].Input.PenOverHeight.Filtered`, `.Status.BadQuality`	Mean of the filtered ΔH_i of the valid pairs. Bad when no pair is valid. (The mean inside and outside levels are computed alongside, for display.)	`Pen.Input.AverageOverHeight.Filtered` [m], `.Status.BadQuality`

SM	Code unit (task)	Reads	Operation	Writes
6	`PenControl` body and `.AlarmHandling` (Task)	`Pen.Input.AverageOverHeight`	Process value of LIC01, and `LevelControlOK := NOT BadQuality`. Alarm `Pen.Alarm.LevelMeasurementLost` while no pair is valid.	`Pen.Status.OverHeight` [m], `Pen.Status.LevelControlOK`
7	`InletSupervisor` body (Task)	`Inlet[i].InletPump.State`, `.Status.EnableLevelControl`, `.Input.MotorVelocity`	Start-up tracking [FD §3.2.1]: if a pump in level control is in START and none is in REMOTE, `LevelTracking := TRUE` and `LevelTrackSpeed` = mean actual speed of the starting pumps. Heartbeat loss [FD Tabell 5]: `LevelHold` while a pump that was in level control has no link or is in HOLD (output frozen at `LevelHoldOutput`); then `LevelResume` for one LIC01 cycle on the mean actual speed of the pumps in level control.	`LevelTracking`, `LevelTrackSpeed`, `LevelHold`, `LevelResume` [rpm] (local)
8	`InletSupervisor.LevelControl` (`PIDController`) → `Util.PID` (Task)	SP `Pen.Command.LevelSetpoint`, PV `Pen.Status.OverHeight`, `Pen.Status.LevelControlOK`, `PenConfig.LevelController.*`, `LevelController.Command.*`, SM 7	Enabled when `PenStart` and at least one pump in level control. Manual when the operator says so, when the level is lost (`NOT LevelControlOK`), on heartbeat loss (hold, resume) or while tracking; the manual output is, in that order, the operator's, `SafeOutput` (limited to Min/MaxOutput), the frozen output, the resume speed or `LevelTrackSpeed`. PI: $Y = K_p\,(e + \frac{1}{T_i}\int e\,dt + T_d\,\dot{e}) + Y_{\text{offset}}$, limited to `MinOutput..MaxOutput`. Manual→auto: `RESET` with $Y_{\text{offset}} = Y_{\text{prev}} - K_p e$ (bumpless).	`GVLMain.LevelController.Output.OUT` [rpm], `LevelController.Input.LimitsActive`, `.Overflow`
9	`Main` → `InletControl[i]` body (Task)	`LevelControllerOut` := `GVLMain.LevelController.Output.OUT`, `InletPump.Status.Manual`, `InletPump.Command.Setpoint`	`InletPumpSpeedSetpoint := Command.Setpoint` in manual, `LevelControllerOut` in auto. One setpoint for all pumps in auto [FD §3.2.2].	`InletPumpSpeedSetpoint` [rpm] (local)
10	`InletControl[i].InletPumpControl` (Task)	`AutoSpeedSetpoint` (= SM 9), `InletPump.State`, `InletPump.Input.MotorVelocity`	Per state (Table 2): REMOTE sends the setpoint; AVAILABLE, START and HOLD send the actual speed; the other states 0. Body: `InternalCommand.VelocitySetpoint := Output.VelocitySetpoint`. Also sets `Status.EnableLevelControl`, `Status.ActualSpeed`.	`Inlet[i].InletPump.Output.VelocitySetpoint`, `Inlet[i].InletPump.InternalCommand.VelocitySetpoint` [rpm]

SM	Code unit (task)	Reads	Operation	Writes
11	`FramoDataProcessing` (OPC_Processing)	`Inlet[i].InletPump.InternalCommand.VelocitySetpoint`; `Framo_n.*`	Out: `Framo_n.SpeedSetpoint.Value := VelocitySetpoint` (REAL→LREAL). In: `InletPump.Input.MotorVelocity := LREAL_TO_REAL(Framo_n.MotorVelocity.Value)`, alarms, heartbeat, state (Table 6, F).	`Framo_n.SpeedSetpoint.Value` [rpm]; `Inlet[i].InletPump.Input.*`, `Inlet[i].InletPump.State`
12	`Datasources_PRG` (DatasourcesTask)	data source variables `Framo_n`	OPC UA client: writes `SpeedSetpoint`, `Start`, `Stop`, `Reset` to the Framo controller's nodes (`VelocitySetpoint` etc. in [FD Tabell 6]) and reads its variables.	Framo controller n

7 Deviations and open issues

Found while tracing the dataflow and comparing it with [FD]. The marks Dn in the tables refer to this list.

	Finding	Consequence / suggestion
D1	**Heartbeat not wired.** `InletControl.Heartbeat` watches the local `HeartbeatToggle`, which is never written; `FramoDataProcessing` writes `InletPump.Input.HeartbeatToggle`. [FD §3.2.1]	`HeartbeatTimeout` is TRUE after 1 s and stays TRUE: the communication alarm is permanent and a real link loss is not detected. **Fixed** 2026-10-15 (in review): one shared bit `Framo_n.HeartbeatToggle`, the PLC sets it while FALSE (after `InletConfig.HeartbeatInterval`), Framo clears it while TRUE; alarm when Framo has not cleared it for `InletConfig.HeartbeatTimeout`. CasFramoSim implements Framo's half; the simulator UI injects *Heartbeat lost*.
D2	`DriveError` alarm wrote `InletPump.Alarm.DriveNotInRemote`.	**Fixed** 2026-10-15 (in review): it writes `Alarm.DriveError`.
D3	**HOLD.** [FD Tabell 5]: on heartbeat loss the LIC01 output shall freeze and, when the link is back, start from the actual pump speed. Oxygenation in that line shall stop.	**Fixed** 2026-10-15 (in review). `InletSupervisor`: when the link to a pump that was in level control is lost, LIC01 is frozen at its output; it stays frozen while the link is down and, once back, while the pump is in HOLD (the operator resets it). When the pump is in REMOTE again (or no longer running), LIC01 runs one cycle on the mean actual speed of the pumps in level control and returns to auto bumplessly. `InletControl` sets `Status.EnableOxyControl` only with a live link and the pump in REMOTE: the permissive for the oxygen system on that line.
D4	**Process value.** [FD §3.2.2]: mean of the per-inlet ΔH_i of the active pairs. Code was: mean of all good inside minus mean of all good outside transmitters.	**Fixed** 2026-10-15 (in review): ΔH_i per pair, filtered per pair (`Butterworth2`), mean of the valid pairs; a lost transmitter drops its whole pair (SM 3–5).
D5	LIC01 did not check `Pen.Input.AverageOverHeight.Status.BadQuality`.	With all transmitters bad the PV was 0 m and LIC01 drove the pumps to `MaxOutput`. **Fixed** 2026-10-15 (in review): the health check uses status bits 1..0 only; with no valid pair, alarm *LevelMeasurementLost* and LIC01 in manual at `SafeOutput` (60 rpm), bumpless back to auto.
D6	[FD Tabell 8]: one parameter set per number of running pumps.	One set only (known, deferred).

	Finding	Consequence / suggestion
D7	[FD Tabell 7]: alarm on speed deviation in REMOTE.	Not implemented.
D8	[FD Tabell 7]: start alarm if the pump does not report START within a preset time after the start command.	Code measured the time *in* START, so a pump that never left AVAILABLE gave no alarm, and a normal START sequence longer than 30 s did. **Fixed** 2026-10-15 (in review): `StartupTransitionTimer` runs while Start is commanded and the pump is still AVAILABLE; `ShutdownTransitionTimer` while Stop is commanded and the pump is still starting, in REMOTE or in HOLD. Framo's own START and STOP sequences are not timed.
D9	`ENUM.InletPumpState` has `Available := 0`, and `FramoDataProcessing` only writes the state when one of the flags is TRUE.	With no OPC UA data the PLC saw AVAILABLE and allowed `Start`. **Fixed** 2026-10-15 (in review): `InletControl` sets `Status.CommunicationOK` once Framo has answered the heartbeat (cleared until the timeout); without it `FramoDataProcessing` reports UNAVAILABLE.
D10	`AnalogSensor.TaskIntervalUs` was a constant 10 000 µs.	**Fixed** 2026-10-15 (in review): `GetTaskIntervalUs` reads the cycle time of the calling task from the runtime (`CmpIecTask`: `IecTaskGetCurrent`, `IecTaskGetInfo3`). `AnalogSensor` and `Butterworth2` read it once; fallback 10 ms (`AnalogSensor`) or the measured call interval (`Butterworth2`) if the runtime gives none (event task). All filter instances are called from `Main` in `Task`.
D11	`ConfigValues`: `PenConfig.LevelController.Ti := 600`, the [FD] start value. The site test used 60 s [DES §7].	Applies on a fresh download only; check the retained value.
D12	`AnalogSensor.Input` is `VAR_IN_OUT`; `ScaleInput` writes the masked word back.	`Input.Raw` on the HMI never shows the status bits. Mask a local copy.
D13	`Pen.Command.LevelSetpoint` is not retained. [FD]: setpoint from the operating mode of the outlet valve (HOLD).	After a restart the setpoint is 0 m.
D14	`PRG.Controllers` (LIC01, oxygen and oxy pump PIDs) is not called by any task.	Dead code with the same output variables as `InletSupervisor`; remove it.

	Finding	Consequence / suggestion
D15	`InletRIO.InputMapping`: `MainCircuitBreakerTripped := InputData[2].2`, `BackupPowerOn := InputData[3].3`. Words 2 and 3 are analog (pressures).	Probably `InputData[4].2` and `InputData[4].3`.